Academic Technical Report

InnoDeploy

A Multi-Tenant DevOps SaaS Platform for CI/CD Automation, Deployment Orchestration, Monitoring & Alerting

Project Type: End-of-Studies Engineering Project (PFE)
Academic Year: 2025 – 2026
Domain: Software Engineering – DevOps

InnoDeploy

April 3, 2026

Contents

1 General Introduction **1**

1.1 Context and Motivation 1

1.2 Project Objectives 1

1.3 Report Structure 2

2 State of the Art **3**

2.1 DevOps and CI/CD Concepts 3

2.1.1 Continuous Integration (CI) 3

2.1.2 Continuous Deployment (CD) 3

2.1.3 Infrastructure as Code (IaC) 3

2.2 Existing Platforms Comparison 4

2.3 Technology Landscape 4

2.3.1 Container Technologies 4

2.3.2 Node.js Ecosystem 4

2.3.3 Modern Frontend Frameworks 4

2.3.4 Message Queues and Workers 5

3 Requirements Analysis **6**

3.1 Actors Identification 6

3.2 Functional Requirements 6

3.2.1 Authentication and Authorisation 6

3.2.2 Project Management 7

3.2.3 CI/CD Pipeline 7

3.2.4 Deployment 7

3.2.5 Host Management 7

3.2.6 Monitoring and Alerting 7

3.2.7 Logging 8

3.2.8 Organisation Settings 8

3.3 Non-Functional Requirements 8

3.4 Use Case Diagram 9

3.5 Detailed Use Cases 10

3.5.1 Use Case: Trigger Pipeline Run 10

3.5.2 Use Case: Register and Login 11

4 System Architecture **12**
4.1 Global Architecture Overview 12
4.2 Technology Stack 13
4.3 Component Architecture 13
4.3.1 Backend Components 13
4.3.2 Frontend Architecture 14
4.4 Multi-Tenancy Architecture 15
4.5 Real-Time Communication Architecture 15

5 Data Modelling **16**
5.1 Class Diagram 16
5.2 Data Model Families 17
5.3 Database Indexing Strategy 18

6 Detailed Design **19**
6.1 Authentication Flow – Login Sequence 19
6.2 Pipeline Execution Flow 20
6.3 Deployment Flow 21
6.4 Webhook-Triggered Pipeline 22
6.5 Token Refresh Flow 22

7 Implementation **23**
7.1 Backend REST API 23
7.1.1 API Endpoint Summary 23
7.1.2 Request Validation with Zod 25
7.1.3 Security Implementation 25
7.2 Worker Subsystem 26
7.2.1 Pipeline Runner 26
7.2.2 Deploy Worker 26
7.2.3 Monitor Worker 26
7.2.4 Log Collector 27
7.3 Frontend Dashboard 27
7.3.1 State Management 27
7.3.2 API Client with Token Refresh 27
7.3.3 Real-Time WebSocket Integration 28
7.4 CLI Tool 28
7.5 Pipeline Configuration File 29

8 User Interface **30**

8.1 Login Page 30
8.2 Registration Page 31
8.3 Dashboard Overview 31
8.4 Projects Page 32
8.5 Key UI Component Summary 33

9 Deployment **34**
9.1 Deployment Diagram 34
9.2 Docker Compose Configuration 34
9.3 Traefik Configuration 35
9.4 Environment Variables 36
9.5 Deployment Strategies Supported 36

10 Testing **38**
10.1 Testing Strategy 38
10.2 Unit Testing 38
10.3 Integration Testing 38
10.4 End-to-End Testing 39
10.5 Pipeline End-to-End Testing 39

11 Conclusion **40**
11.1 Summary of Contributions 40
11.2 Technical Achievements 41
11.3 Limitations 41
11.4 Future Work 41
11.5 Final Remarks 42

A Project Directory Structure **43**

B Glossary **44**

List of Figures

3.1 Global Use Case Diagram 9

4.1 Global System Architecture 12
4.2 Backend Layered Architecture 14
4.3 Frontend Component Architecture 14
4.4 Real-Time Communication via Redis Pub/Sub 15

5.1 Complete UML Class Diagram 17

6.1 Login Authentication Sequence Diagram 19
6.2 Pipeline Execution Sequence Diagram 20
6.3 Deployment Execution Sequence Diagram 21
6.4 Webhook-Triggered Pipeline Sequence Diagram 22
6.5 Token Refresh Sequence Diagram 22

8.1 Login Page – Email/Password and OAuth Authentication 30
8.2 Registration Page – Account Creation with Organisation 31
8.3 Projects Page – Project List with Status 32

9.1 UML Deployment Diagram 34

10.1 Testing Pyramid 38

List of Tables

2.1 Comparison of Existing CI/CD Platforms 4

3.1 System Actors and Their Roles 6
3.2 Non-Functional Requirements 8
3.3 Use Case: Trigger Pipeline Run 10
3.4 Use Case: User Registration 11

4.1 Technology Stack 13

5.1 Key Database Indexes 18

7.1 Complete API Endpoint Reference 23

8.1 Dashboard UI Components 33

9.1 Docker Compose Services 35
9.2 Key Environment Variables 36
9.3 Deployment Strategies 37

Chapter 1

General Introduction

1.1 Context and Motivation

The modern software development landscape demands rapid, reliable, and repeatable delivery of applications to production environments. DevOps practices have emerged as the standard methodology for bridging the gap between software development (Dev) and IT operations (Ops), emphasizing automation, continuous integration, and continuous deployment (CI/CD).

However, existing CI/CD platforms such as Jenkins, GitLab CI, or GitHub Actions present significant challenges for small-to-medium teams and organisations:

- **Complexity**: Configuration of pipelines requires deep infrastructure knowledge.
- **Fragmentation**: Monitoring, alerting, deployment, and secret management are often handled by separate tools.
- **Cost**: Enterprise solutions carry prohibitive licensing costs.
- **Self-hosting burden**: Teams must maintain CI/CD infrastructure themselves.

InnoDeploy addresses these challenges by offering a unified, multi-tenant SaaS platform that consolidates CI/CD pipeline execution, deployment orchestration across multiple strategies (rolling, blue-green, canary, recreate), real-time infrastructure monitoring, log aggregation, alerting, and secret management into a single cohesive system.

1.2 Project Objectives

The primary objectives of this project are:

1. Design and implement a **multi-tenant architecture** supporting organisation-scoped resource isolation with role-based access control (RBAC).
2. Build a **CI/CD pipeline engine** capable of executing user-defined build steps in isolated Docker containers with configurable timeout and retry policies.
3. Implement four **deployment strategies**: rolling, blue-green, canary, and recreate — executed via SSH on user-managed hosts.

4. Develop a **real-time monitoring subsystem** collecting CPU, memory, disk, network, and latency metrics with configurable alert thresholds.
5. Provide a **web dashboard** for visual management of all platform features.
6. Provide a **command-line interface (CLI)** for terminal-based workflows.
7. Ensure the platform is **production-ready** with Docker Compose orchestration under a Traefik reverse proxy with automatic TLS.

1.3 Report Structure

This report is organised as follows:

Chapter 2 – State of the Art
Reviews existing DevOps platforms, CI/CD concepts, and container orchestration technologies.

Chapter 3 – Requirements Analysis
Defines functional and non-functional requirements using UML use case diagrams.

Chapter 4 – System Architecture
Presents the global architecture, component interactions, and technology stack.

Chapter 5 – Data Modelling
Details the database schema using UML class diagrams.

Chapter 6 – Detailed Design
Provides sequence diagrams for key workflows.

Chapter 7 – Implementation
Covers the backend API, frontend dashboard, CLI, and worker subsystems.

Chapter 8 – User Interface
Showcases the dashboard screens.

Chapter 9 – Deployment
Describes the Docker Compose infrastructure and Traefik configuration.

Chapter 10 – Testing
Covers unit, integration, and end-to-end testing.

Chapter 11 – Conclusion
Summarizes contributions and outlines future work.

Chapter 2

State of the Art

2.1 DevOps and CI/CD Concepts

2.1.1 Continuous Integration (CI)

Continuous Integration is the practice of merging code changes into a shared repository frequently, with automated build and test execution on each merge. The goal is to detect integration errors early and reduce the feedback loop for developers.

2.1.2 Continuous Deployment (CD)

Continuous Deployment extends CI by automatically deploying every change that passes the test suite to production environments, eliminating manual release gates and enabling rapid iteration.

2.1.3 Infrastructure as Code (IaC)

IaC treats infrastructure configuration as version-controlled code, enabling reproducible environments and reducing configuration drift. Tools like Docker, Terraform, and Ansible exemplify this paradigm.

2.2 Existing Platforms Comparison

Table 2.1: Comparison of Existing CI/CD Platforms

Feature	**Jenkins**	**GitLab CI**	**GitHub Actions**	**InnoDeploy**
Self-hosted	Yes	Yes	No	Yes
Multi-tenant	No	Limited	No	**Yes**
Built-in monitoring	No	Limited	No	**Yes**
Deployment strategies	Plugin	Limited	No	**4 strategies**
Log aggregation	Plugin	Basic	Basic	**Built-in**
Alert rules engine	No	No	No	**Yes**
Secret management	Plugin	Yes	Yes	**Yes**
CLI tool	Yes	Yes	Yes	**Yes**
WebSocket real-time	No	Limited	No	**Yes**
SSH deployment	Plugin	Yes	No	**Yes**
Cost	Free (OSS)	Freemium	Freemium	**Free (OSS)**

2.3 Technology Landscape

2.3.1 Container Technologies

Docker enables application packaging into lightweight, portable containers. Docker Compose orchestrates multi-container applications. Traefik serves as a cloud-native reverse proxy with automatic service discovery.

2.3.2 Node.js Ecosystem

Express.js provides a minimal, flexible web framework for building REST APIs. The event-driven, non-blocking I/O model of Node.js is well-suited for real-time applications with WebSocket support.

2.3.3 Modern Frontend Frameworks

Next.js 15 offers server-side rendering, static site generation, and API routes within a React-based framework. Combined with TypeScript, Tailwind CSS, and component libraries like Shadcn UI, it enables rapid development of responsive dashboards.

2.3.4 Message Queues and Workers

BullMQ provides Redis-backed job queues for reliable asynchronous task processing, essential for pipeline execution and deployment orchestration where jobs may take minutes to complete.

Chapter 3

Requirements Analysis

3.1 Actors Identification

The system identifies four actor roles with hierarchical permissions:

Table 3.1: System Actors and Their Roles

Actor	Description
Owner	Creates the organisation. Full administrative privileges including organisation deletion, billing management, and member management.
Admin	Manages projects, hosts, pipelines, members, and settings. Cannot delete the organisation.
Developer	Creates projects, triggers deployments and pipelines, manages environments and secrets.
Viewer	Read-only access to projects, pipelines, logs, metrics, and alerts.

3.2 Functional Requirements

3.2.1 Authentication and Authorisation

- FR-01: Users shall register with name, email, password, and optional organisation name.
- FR-02: Users shall authenticate via email/password, Google OAuth, or GitHub OAuth.
- FR-03: The system shall issue JWT access tokens (short-lived) and refresh tokens (7-day TTL stored in Redis).
- FR-04: Role-based access control shall enforce permissions at the route level.
- FR-05: Users shall be able to reset passwords via email verification.

3.2.2 Project Management

- FR-06: Users shall create, read, update, and delete projects linked to Git repositories.
- FR-07: Projects shall support multiple environments (e.g., staging, production) with independent configurations.
- FR-08: Each environment shall have encrypted secret storage (AES encryption).
- FR-09: Project creation shall respect organisation plan limits.

3.2.3 CI/CD Pipeline

- FR-10: Users shall trigger pipeline runs via the dashboard, CLI, or Git webhooks.
- FR-11: Pipelines shall execute user-defined steps in isolated Docker containers.
- FR-12: Each step shall support configurable image, command, timeout, and retry count.
- FR-13: Pipeline progress shall stream in real-time via Server-Sent Events (SSE).
- FR-14: Users shall cancel running pipelines.
- FR-15: Pipeline configuration shall be definable via `.innodeploy.yml` files.

3.2.4 Deployment

- FR-16: Users shall trigger deployments to registered hosts.
- FR-17: The system shall support four deployment strategies: rolling, blue-green, canary, and recreate.
- FR-18: Users shall rollback deployments to previous versions.
- FR-19: Deployments shall execute via SSH on user-managed hosts.

3.2.5 Host Management

- FR-20: Users shall register remote hosts with SSH credentials.
- FR-21: The system shall test SSH connectivity and retrieve OS/Docker information.
- FR-22: Hosts shall display real-time CPU, memory, disk usage, and container status.

3.2.6 Monitoring and Alerting

- FR-23: The system shall collect CPU, memory, disk, network, latency, and uptime metrics.
- FR-24: Metrics shall stream in real-time via WebSocket.

- FR-25: Configurable alert rules shall trigger on threshold violations (CPU, memory, latency, availability, disk, certificate expiry).
- FR-26: Alerts shall be dispatched via email, Slack, Discord, Expo push notifications, and custom webhooks.
- FR-27: Users shall acknowledge and resolve alerts.

3.2.7 Logging

- FR-28: The system shall aggregate container logs with full-text search.
- FR-29: Logs shall support filtering by level, environment, source, and container.
- FR-30: Log retention shall be configurable with TTL-based auto-expiration.

3.2.8 Organisation Settings

- FR-31: Admins shall configure notification channels, Docker registry, and Git provider settings.
- FR-32: Admins shall manage API keys for programmatic access.
- FR-33: The system shall maintain a full audit log of all administrative actions with 90-day retention.

3.3 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Category	Requirement
Performance	API response time < 200ms for CRUD operations. Pipeline step timeout configurable (default 10 min).
Scalability	Multi-tenant isolation allows horizontal scaling of workers. Configurable pipeline runner concurrency.
Security	Passwords hashed with bcrypt (12 salt rounds). Secrets encrypted with AES. JWT with server-side revocation. Rate limiting on auth endpoints.
Reliability	MongoDB retry logic (5 attempts). Redis reconnection with exponential backoff. Graceful shutdown for all services.
Usability	Responsive dashboard supporting light, dark, and system themes. Internationalisation support (English, French, Arabic).
Maintainability	Modular architecture with clear separation of concerns. Zod schema validation at API boundaries.

3.4 Use Case Diagram

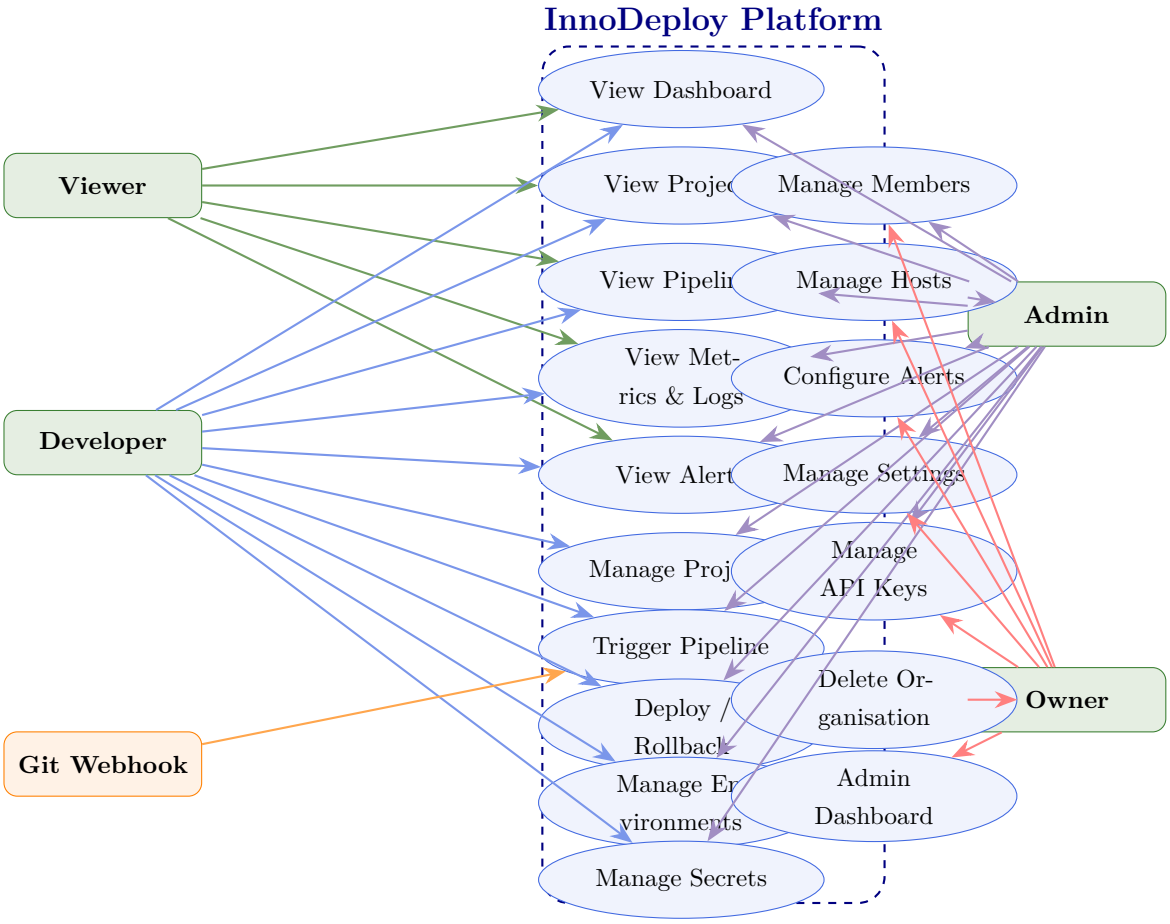

Figure 3.1: Global Use Case Diagram

3.5 Detailed Use Cases

3.5.1 Use Case: Trigger Pipeline Run

Table 3.3: Use Case: Trigger Pipeline Run

Use Case	Trigger Pipeline Run
Actor	Developer, Admin, Owner, Git Webhook
Precondition	Project exists; user has developer+ role; or valid webhook signature
Main Flow	1. Actor selects project and provides pipeline parameters (version, strategy, branch, environment, steps). 2. System validates the request via Zod schema. 3. System creates a Pipeline document with status "pending". 4. System enqueues the job to the Redis pipeline queue. 5. Pipeline runner worker dequeues and executes steps sequentially in Docker containers. 6. System streams progress via SSE and WebSocket. 7. On completion, system updates Pipeline status to "success" or "failed".
Postcondition	Pipeline run recorded with step-level outputs and durations.
Alternative	A1: Validation fails → return 400 with field errors. A2: User cancels → set status to "cancelled", skip remaining steps. A3: Step timeout → mark step as "failed", proceed based on retry config.

3.5.2 Use Case: Register and Login

Table 3.4: Use Case: User Registration

Use Case	User Registration
Actor	Unregistered User
Precondition	Email not already registered
Main Flow	1. User provides name, email, password, and optional organisation name. 2. System validates input via Zod schema. 3. System hashes password with bcrypt (12 rounds). 4. System creates User document. 5. If organisation name provided, system creates Organisation and assigns user as owner. 6. System issues JWT access token and refresh token.
Postcondition	User account created; tokens stored client-side.
Alternative	A1: Email already exists → return 409 conflict. A2: Rate limit exceeded → return 429.

Chapter 4

System Architecture

4.1 Global Architecture Overview

InnoDeploy follows a **microservices-inspired monorepo architecture** with clear separation between the API server, background workers, real-time gateway, and frontend client.

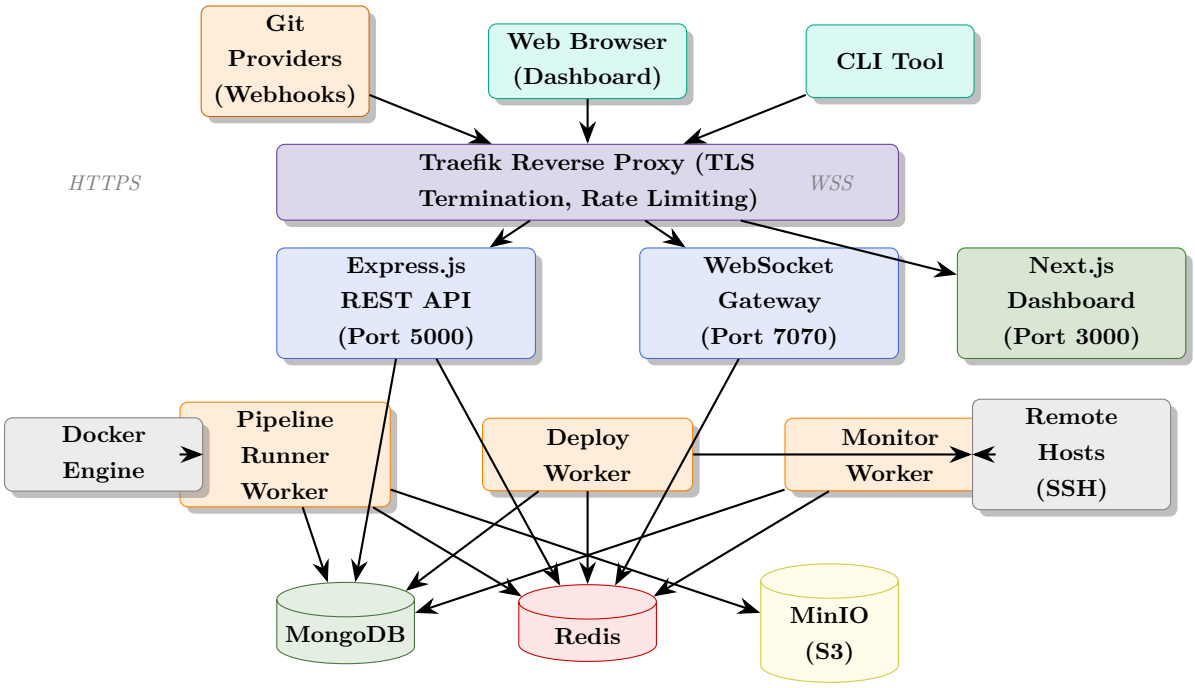

Figure 4.1: Global System Architecture

4.2 Technology Stack

Table 4.1: Technology Stack

Layer	Technology	Role
Frontend	Next.js 15	Server-side rendering, routing, and API integration
	React 19	Component-based UI framework
	TypeScript 5.5	Static type safety
	Tailwind CSS 3.4	Utility-first CSS framework
	Zustand 4.5	Lightweight state management
	React Query 5.51	Server state caching and synchronization
	Recharts 2.15	Data visualization charts
	xterm.js 5.3	Terminal emulator for log viewing
Backend	Express.js 4.21	HTTP REST API framework
	Mongoose 8.5	MongoDB ODM with schema validation
	Redis 4.7	Caching, token storage, pub/sub, job queues
	BullMQ 5.71	Persistent job queue management
	Dockerode 4.0	Docker Engine API client
	SSH2 1.17	SSH client for remote host access
	Zod 4.3	Runtime schema validation
	JSON Web Token 9.0	Stateless authentication tokens
Infrastructure	Docker & Compose	Container orchestration
	Traefik 3.1	Reverse proxy with auto-TLS
	MongoDB 7	Document database
	MinIO	S3-compatible object storage
CLI	Commander.js	CLI argument parsing and subcommands

4.3 Component Architecture

4.3.1 Backend Components

The backend is organised into the following layers:

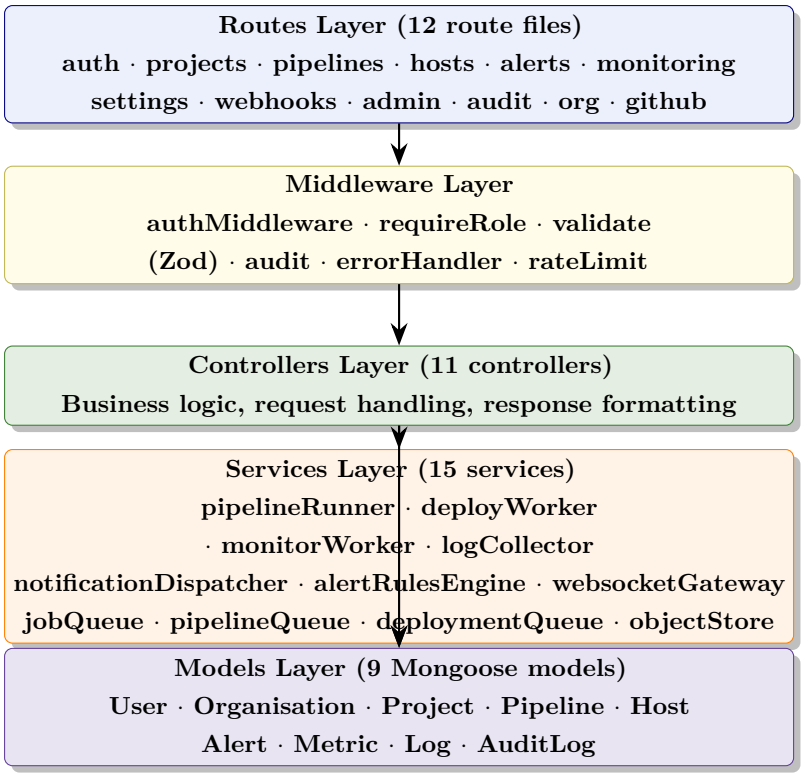

Figure 4.2: Backend Layered Architecture

4.3.2 Frontend Architecture

The Next.js dashboard follows a page-based routing architecture:

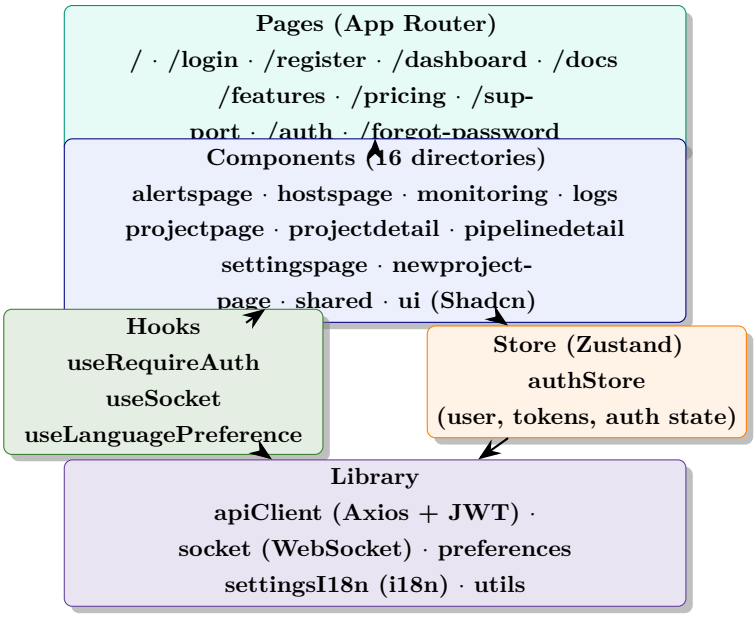

Figure 4.3: Frontend Component Architecture

4.4 Multi-Tenancy Architecture

InnoDeploy implements **organisation-level multi-tenancy** where all resources are scoped to an organisation:

- Each **User** belongs to exactly one **Organisation**.
- Each **Project**, **Host**, and **AuditLog** is scoped to an `organisationId`.
- **Pipelines**, **Alerts**, **Metrics**, and **Logs** are scoped via their parent `projectId`.
- The middleware chain validates organisation membership before allowing access.
- Plan limits (maxProjects, maxDeploysPerMonth, maxMembers) are enforced at the organisation level.

4.5 Real-Time Communication Architecture

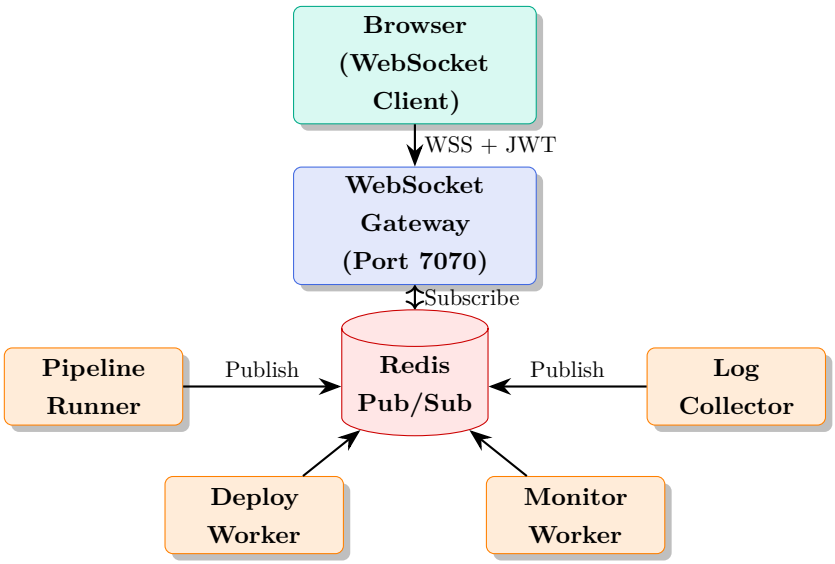

Figure 4.4: Real-Time Communication via Redis Pub/Sub

The WebSocket gateway supports the following channel subscriptions:

- `monitoring:stream`, `monitoring:project:{id}` – real-time metrics
- `pipeline:events`, `pipeline:logs:{id}` – pipeline progress
- `deploy:events` – deployment status updates
- `logs:stream`, `logs:project:{id}` – container log streaming

Chapter 5

Data Modelling

5.1 Class Diagram

The following UML class diagram represents the complete data model of the InnoDeploy platform. The system uses MongoDB as its primary database with Mongoose ODM for schema enforcement.

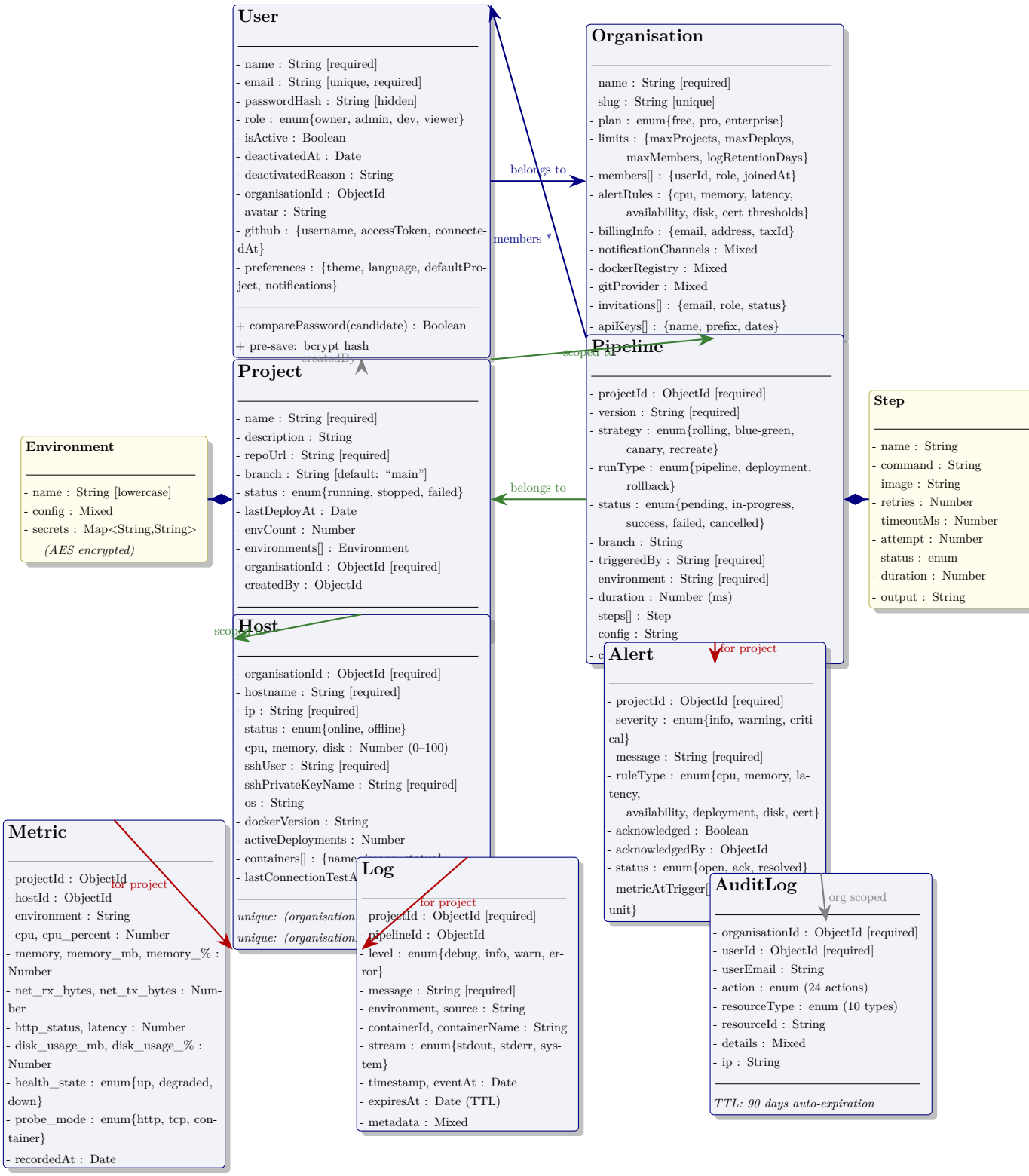

Figure 5.1: Complete UML Class Diagram

5.2 Data Model Families

The models can be categorised into three domain families:

Identity Domain

`User`, `Organisation` – Authentication, authorisation, and multi-tenancy.

Delivery Domain

`Project`, `Pipeline`, `Host` – CI/CD pipeline execution and deployment orchestration.

Observability Domain

`Alert`, `Metric`, `Log`, `AuditLog` – Monitoring, alerting, logging, and audit trail.

5.3 Database Indexing Strategy

Table 5.1: Key Database Indexes

Collection	Index	Purpose
Project	(organisationId, name) unique	Prevent duplicate project names per org
Host	(organisationId, hostname) unique	Prevent duplicate hostnames per org
Host	(organisationId, ip) unique	Prevent duplicate IPs per org
AuditLog	(organisationId, createdAt desc)	Fast audit log queries
AuditLog	expiresAt TTL	90-day auto-expiration
Log	(projectId, eventAt desc)	Chronological log queries
Log	(projectId, level, eventAt desc)	Level-filtered log queries
Log	text(message, source, containerName)	Full-text search
Log	expiresAt TTL	Configurable log retention

Chapter 6

Detailed Design

6.1 Authentication Flow – Login Sequence

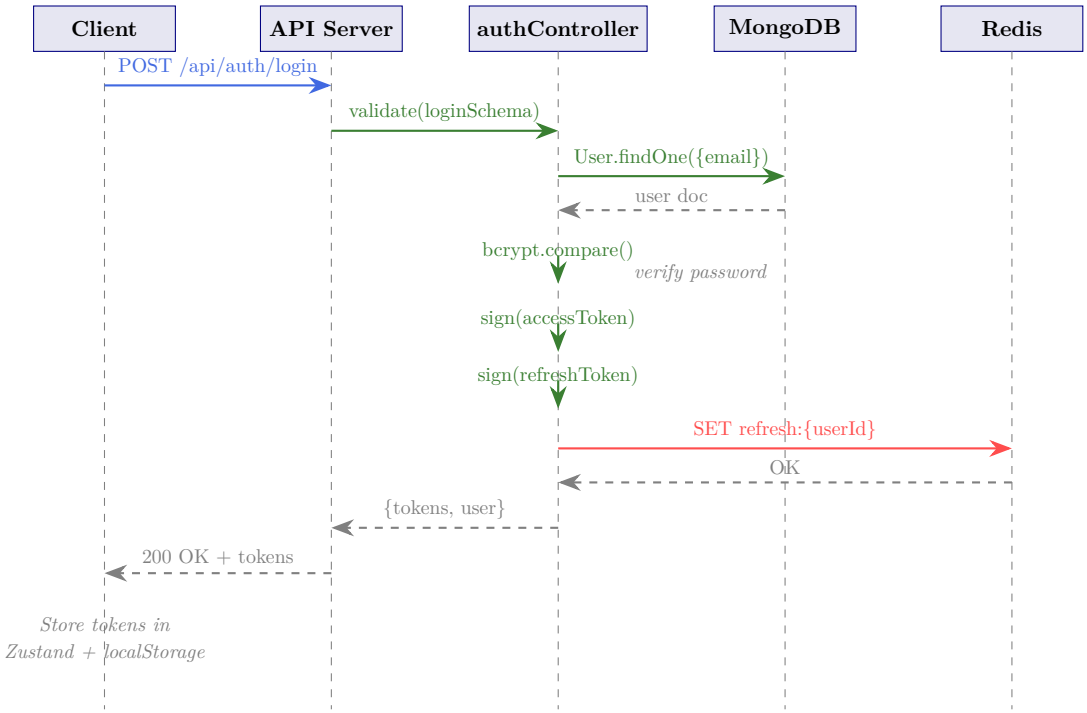

Figure 6.1: Login Authentication Sequence Diagram

6.2 Pipeline Execution Flow

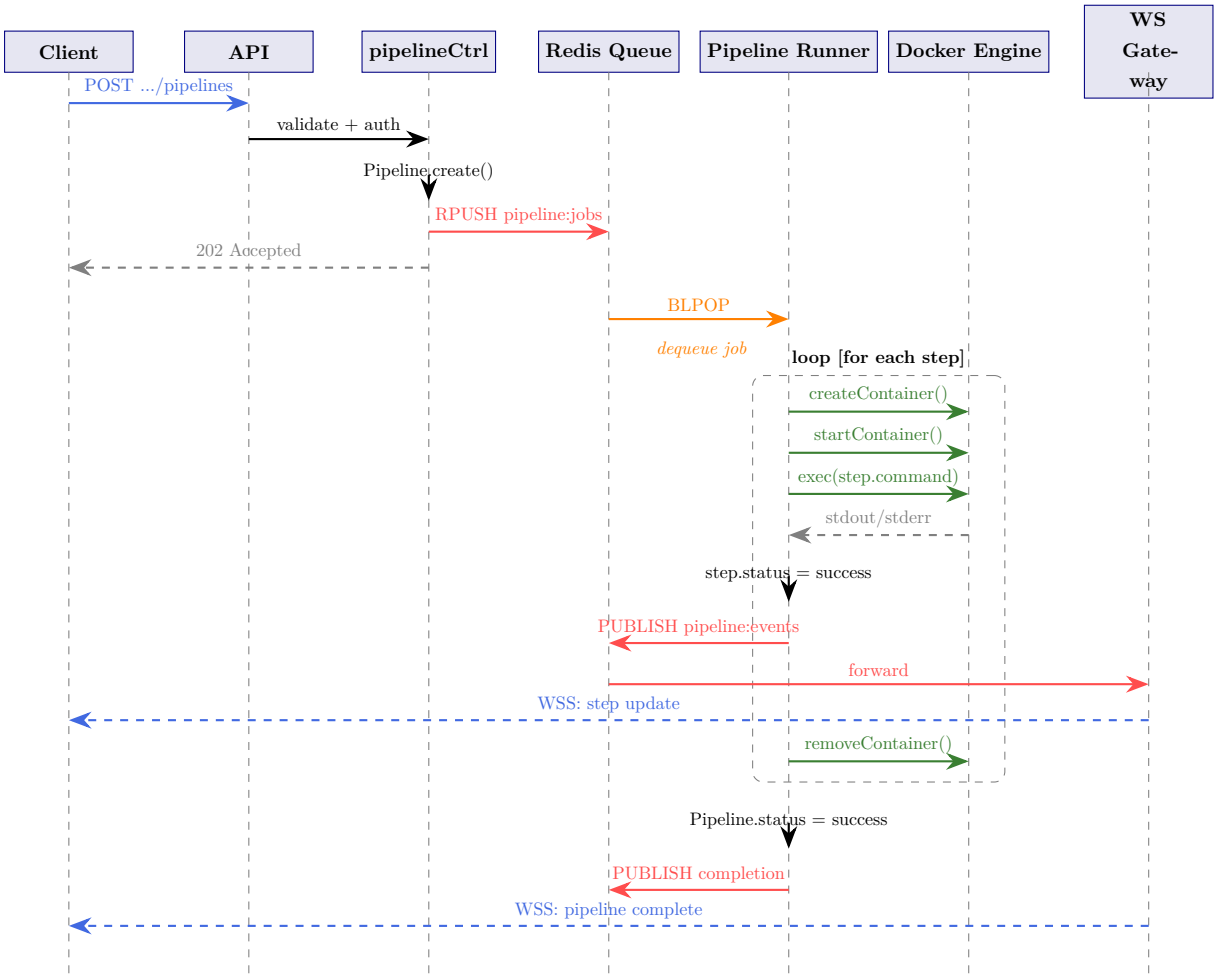

Figure 6.2: Pipeline Execution Sequence Diagram

6.3 Deployment Flow

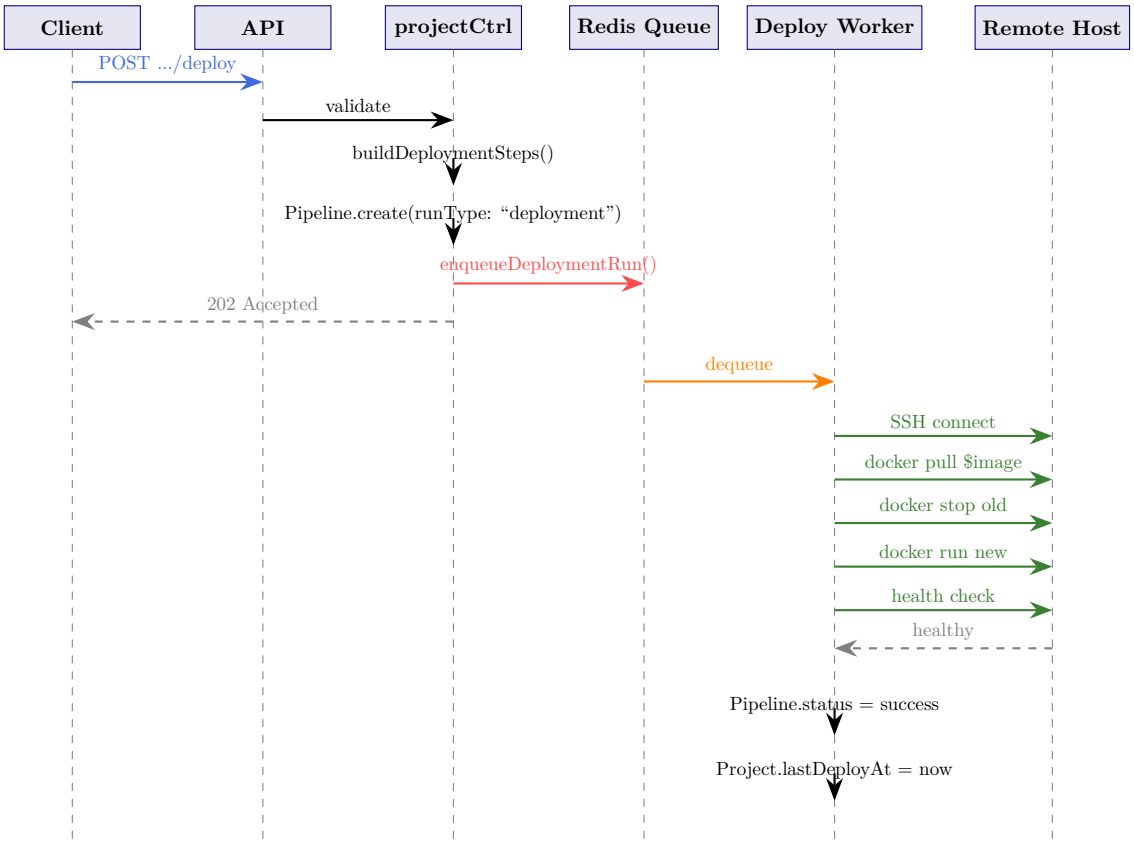

Figure 6.3: Deployment Execution Sequence Diagram

6.4 Webhook-Triggered Pipeline

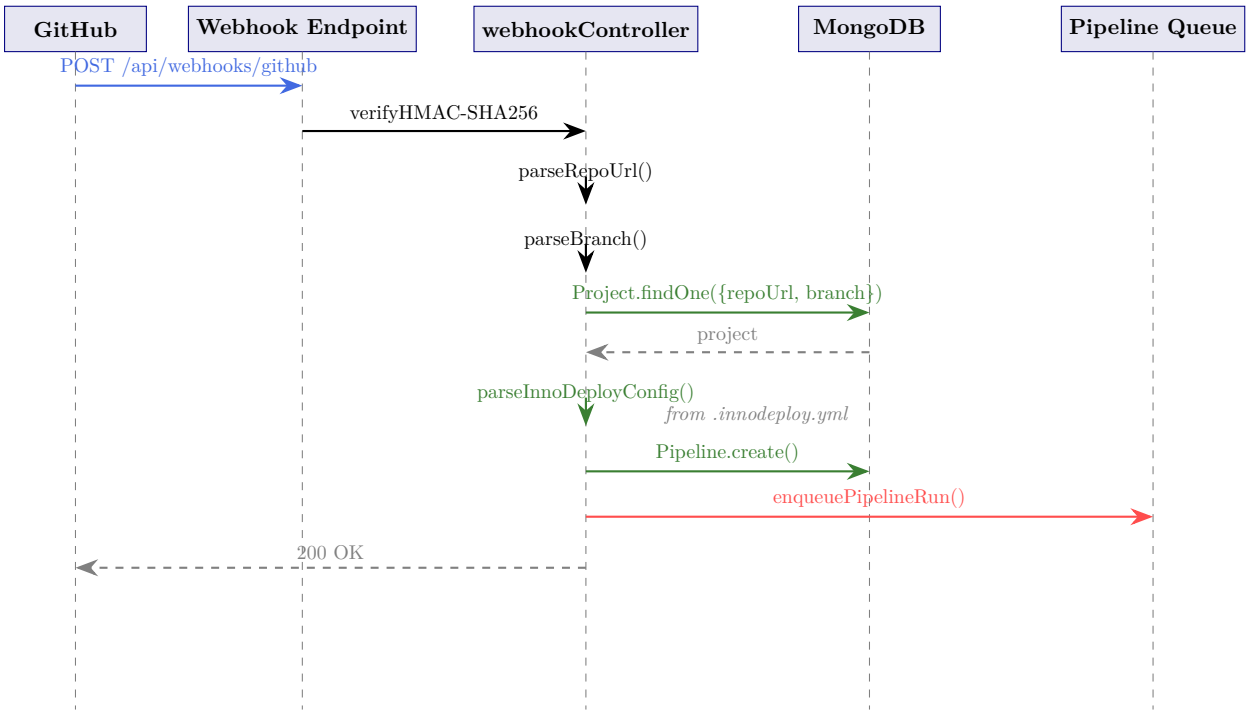

Figure 6.4: Webhook-Triggered Pipeline Sequence Diagram

6.5 Token Refresh Flow

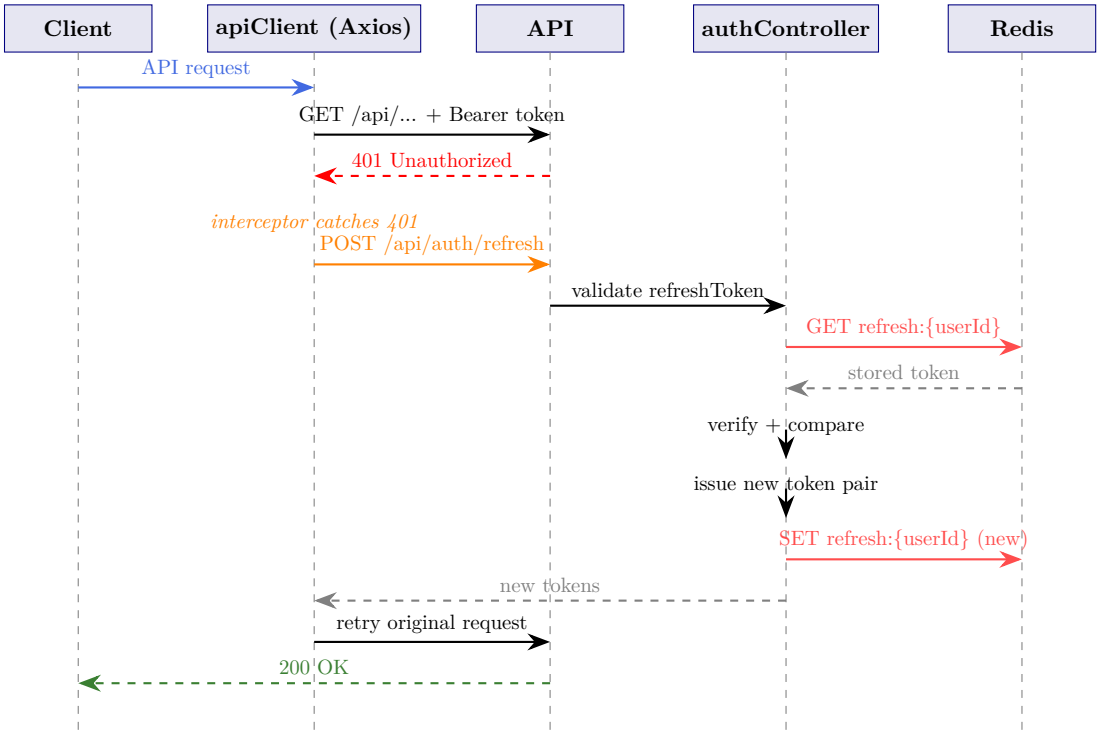

Figure 6.5: Token Refresh Sequence Diagram

Chapter 7

Implementation

7.1 Backend REST API

7.1.1 API Endpoint Summary

The backend exposes approximately 60 RESTful endpoints organised across 12 route files:

Table 7.1: Complete API Endpoint Reference

Method	Path	Description
	Authentication	
POST	/api/auth/register	User registration
POST	/api/auth/login	Email/password login
POST	/api/auth/refresh	Token refresh
POST	/api/auth/logout	Revoke refresh token
POST	/api/auth/forgot-password	Initiate password reset
POST	/api/auth/reset-password	Complete password reset
GET	/api/auth/google	Google OAuth initiation
GET	/api/auth/google/callback	Google OAuth callback
GET	/api/auth/github	GitHub OAuth initiation
GET	/api/auth/github/callback	GitHub OAuth callback
	Projects	
GET	/api/projects	List all projects
POST	/api/projects	Create project
GET	/api/projects/:id	Get project details
PATCH	/api/projects/:id	Update project
DELETE	/api/projects/:id	Delete project (cascade)
POST	/api/projects/:id/envs	Create environment
PATCH	/api/projects/:id/envs/:env	Update environment

Continued on next page

Method	Path	Description
DELETE	/api/projects/:id/envs/:env	Delete environment
POST	/api/projects/:id/envs/:env/secrets	Upsert secret
POST	/api/projects/:id/deploy	Trigger deployment
POST	/api/projects/:id/rollback	Rollback deployment
GET	/api/projects/:id/deploy/history	Deployment history
Pipelines		
POST	/api/projects/:id/pipelines	Trigger pipeline
GET	/api/projects/:id/pipelines	List runs
GET	/api/pipelines/:runId	Get run details
POST	/api/pipelines/:runId/cancel	Cancel running pipeline
GET	/api/pipelines/:runId/logs/:stage	Get stage logs
GET	/api/pipelines/:runId/stream	SSE stream
Hosts		
GET	/api/hosts	List hosts
POST	/api/hosts	Create host
GET	/api/hosts/:id	Get host
PATCH	/api/hosts/:id	Update host
DELETE	/api/hosts/:id	Remove host
POST	/api/hosts/test-connection	Test draft SSH
POST	/api/hosts/:id/test-connection	Test existing SSH
Monitoring		
GET	/api/projects/:id/metrics	Get metrics
GET	/api/projects/:id/logs	Get logs
GET	/api/projects/:id/status	Health status
Alerts		
GET	/api/alerts	List alerts
POST	/api/alerts	Create alert
PATCH	/api/alerts/:id/acknowledge	Acknowledge
PATCH	/api/alerts/:id/resolve	Resolve
DELETE	/api/alerts/:id	Delete alert
GET	/api/alerts/rules	Get rules
PUT	/api/alerts/rules	Update rules
POST	/api/alerts/rules/test-notification	Test notification

Continued on next page

Method	Path	Description
	Webhooks	
POST	/api/webhooks/github	GitHub webhook
POST	/api/webhooks/gitlab	GitLab webhook
POST	/api/webhooks/bitbucket	Bitbucket webhook

7.1.2 Request Validation with Zod

All incoming requests are validated at the route level using Zod schemas. Example:

```
const triggerPipelineSchema = z.object({
  version:     z.string().min(1).max(50),
  strategy:    z.enum(["rolling","blue-green","canary","
     recreate"])
                .optional(),
  branch:      z.string().max(100).optional(),
  environment: z.string().min(1).max(50),
  config:      z.string().max(50000).optional(),
  steps:       z.array(z.object({
    name:      z.string().min(1).max(100),
    command:   z.string().min(1).max(5000),
    image:     z.string().max(200).optional(),
    retries:   z.number().int().min(0).max(5).optional(),
    timeoutMs: z.number().int().min(1000).optional()
  })).max(20).optional()
});
```

Listing 7.1: Zod Schema for Pipeline Trigger

7.1.3 Security Implementation

- **Password hashing**: bcrypt with 12 salt rounds (pre-save hook on User model).
- **JWT**: Access token (short-lived) + Refresh token (7-day, stored in Redis for server-side revocation).
- **Secret encryption**: Project environment secrets encrypted with AES before MongoDB storage.
- **Rate limiting**: Auth endpoints limited to 20 requests per 15 minutes; password reset limited to 5 per 15 minutes; webhooks limited to 30 per minute.
- **Security headers**: Helmet.js middleware applies CSP, HSTS, X-Frame-Options, etc.

- **CORS**: Origin validation against configured `CLIENT_URL`.
- **Webhook verification**: HMAC-SHA256 for GitHub, token verification for GitLab, signature verification for Bitbucket.

7.2 Worker Subsystem

7.2.1 Pipeline Runner

The pipeline runner is the core execution engine:

1. Polls the Redis pipeline queue using `BLPOP` (blocking pop).
2. For each step, creates an isolated Docker container from the specified image (default: `node:20-alpine`).
3. Clones the Git repository into the container workspace.
4. Executes the step command, capturing stdout and stderr.
5. Supports configurable timeouts (default 10 minutes) and retries per step.
6. Publishes progress events to Redis channels for real-time streaming.
7. On completion, updates the Pipeline document with final status and duration.
8. Concurrency is configurable via `PIPELINE_RUNNER_CONCURRENCY` (default: 1).

7.2.2 Deploy Worker

The deploy worker handles deployment orchestration:

1. Dequeues deployment jobs from the Redis deployment queue.
2. Establishes SSH connection to the target host using SSH2.
3. Executes deployment steps: image pull, container stop, container start, health check.
4. Supports four strategies: **rolling** (incremental replacement), **blue-green** (stack switch), **canary** (gradual traffic shift), and **recreate** (full restart).
5. Updates Pipeline status and Project deployment timestamp on completion.

7.2.3 Monitor Worker

The monitoring worker continuously collects infrastructure metrics:

1. Periodically polls Docker containers for CPU, memory, network, and disk metrics.
2. Executes health probes (HTTP, TCP, or container-based) to determine service availability.
3. Stores metrics in the Metric collection.
4. Publishes real-time metric updates to Redis monitoring channels.

5. Feeds data to the alert rules engine for threshold evaluation.

7.2.4 Log Collector

1. Aggregates Docker container logs via the Docker API.
2. Stores logs in the Log collection with structured metadata.
3. Publishes log entries to Redis log channels for real-time streaming.
4. Supports TTL-based auto-expiration for storage management.

7.3 Frontend Dashboard

7.3.1 State Management

The dashboard uses **Zustand** for global auth state and **React Query (TanStack Query)** for server state:

```
const useAuthStore = create((set) => ({
  user: null,
  accessToken: null,
  refreshToken: null,
  isAuthenticated: false,
  setAuth: (data) => set({
    user: data.user,
    accessToken: data.accessToken,
    refreshToken: data.refreshToken,
    isAuthenticated: true
  }),
  clearAuth: () => set({
    user: null, accessToken: null,
    refreshToken: null, isAuthenticated: false
  }),
  hydrate: () => { /* restore from localStorage */ }
}));
```

Listing 7.2: Zustand Auth Store

7.3.2 API Client with Token Refresh

The Axios-based API client implements automatic token refresh:

- Request interceptor injects the Bearer token from the auth store.
- Response interceptor catches 401 errors and attempts token refresh.

- On successful refresh, retries the original request transparently.
- On refresh failure, clears auth state and redirects to login.

7.3.3 Real-Time WebSocket Integration

The `useSocket` hook manages WebSocket connections:

- Connects to the WebSocket gateway with JWT authentication.
- Supports subscription to project-scoped monitoring channels.
- Handles reconnection with exponential backoff.
- Routes incoming messages to the appropriate UI components.

7.4 CLI Tool

The `innodeploy` CLI provides terminal-based access to all major platform features:

```
# Authenticate
innodeploy login

# Project management
innodeploy projects list
innodeploy projects create my-app

# Deployment
innodeploy deploy my-app --env production
innodeploy rollback my-app --env production

# Pipeline
innodeploy pipeline trigger my-app
innodeploy pipeline status <runId>

# Real-time logs
innodeploy logs my-app --env production --follow

# Scaffold config
innodeploy init
```

Listing 7.3: CLI Command Examples

The CLI stores credentials in `~/.innodeploy-cli/config.json` and implements automatic token refresh on 401 responses, mirroring the dashboard's behaviour.

7.5 Pipeline Configuration File

Projects can define their CI/CD pipeline using a `.innodeploy.yml` file:

```yaml
name: my-app-pipeline
strategy: rolling
environment: production
trigger:
  branch: main
steps:
  - name: Install dependencies
    command: npm ci
    image: node:20-alpine
    timeoutMs: 300000
  - name: Run tests
    command: npm test
    image: node:20-alpine
    retries: 2
  - name: Build
    command: npm run build
    image: node:20-alpine
  - name: Deploy
    command: ./scripts/deploy.sh
    timeoutMs: 600000
notifications:
  onSuccess: true
  onFailure: true
```

Listing 7.4: .innodeploy.yml Configuration Example

Chapter 8

User Interface

The InnoDeploy dashboard is built with Next.js 15, React 19, and Tailwind CSS, featuring a modern dark-themed design with cyan accent colour (`#00D4AA`). The interface supports three themes (light, dark, system) and three languages (English, French, Arabic).

8.1 Login Page

The login page provides multiple authentication methods: email/password, Google OAuth, and GitHub OAuth.

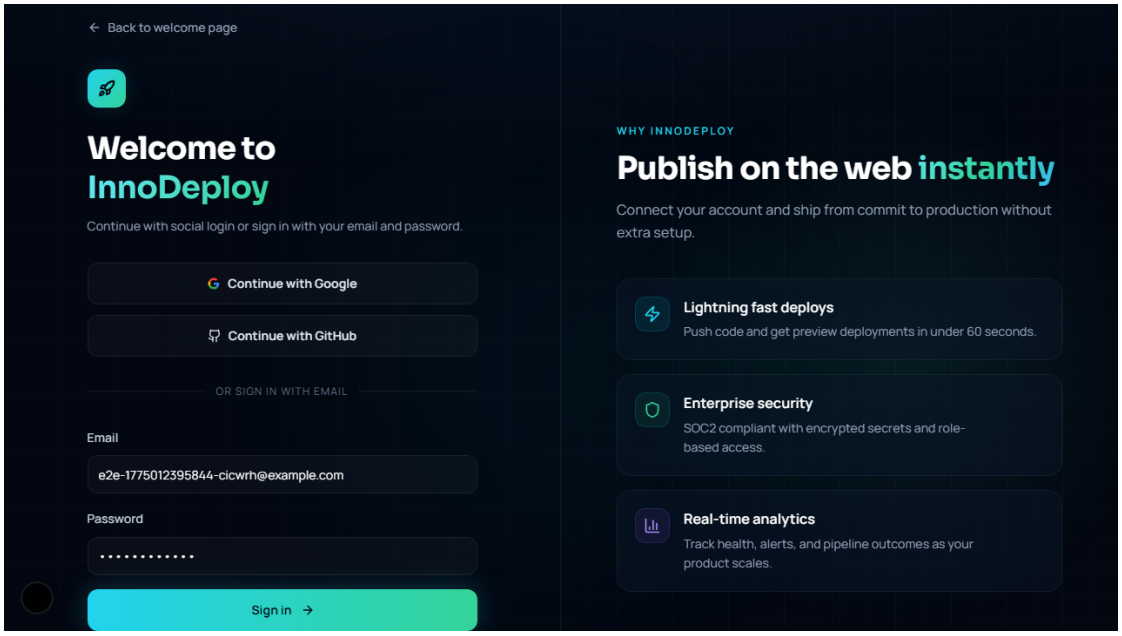

Figure 8.1: Login Page – Email/Password and OAuth Authentication

Key features visible:

- Social login buttons (Google, GitHub) prominently displayed.
- Email and password form fields with validation.
- Right panel showcasing platform value propositions: *Lightning fast deploys*, *Enterprise security*, and *Real-time analytics*.

- Theme toggle in the bottom-left corner.

8.2 Registration Page

New users register with name, email, password, and an optional organisation name:

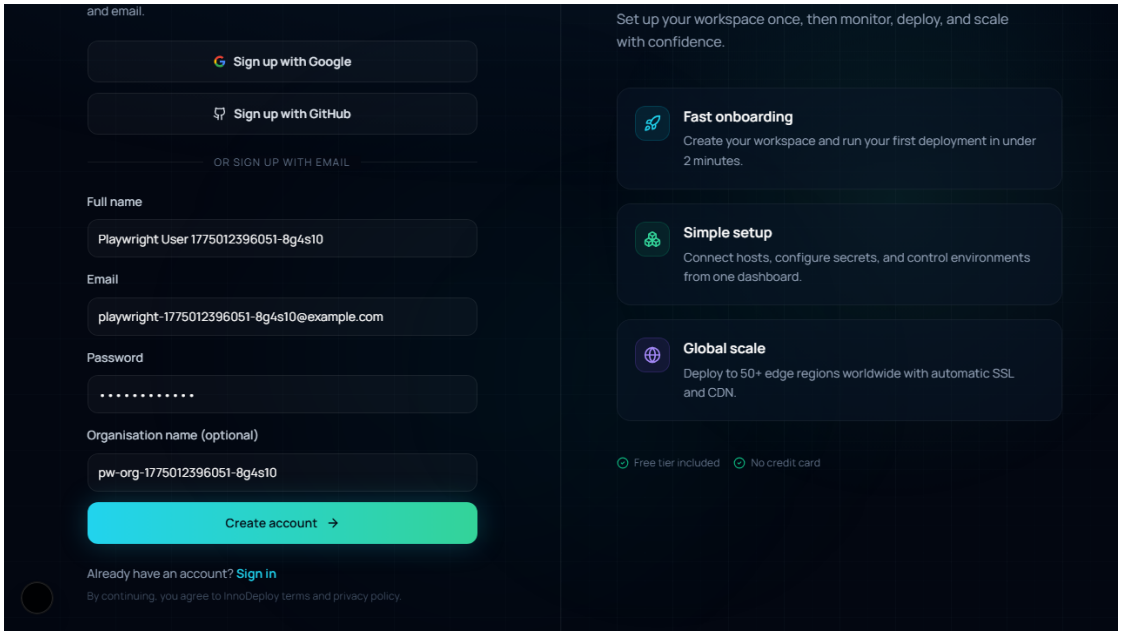

Figure 8.2: Registration Page – Account Creation with Organisation

Key features:

- Social registration options (Google, GitHub).
- Organisation name field allows automatic workspace creation.
- "Free tier included" and "No credit card" trust indicators.
- Right panel highlights: *Fast onboarding*, *Simple setup*, *Global scale*.

8.3 Dashboard Overview

The main dashboard page provides a high-level summary of the organisation's resources:

- **Stats cards**: Total projects, active deployments, hosts online, open alerts.
- **14-day trend charts**: New users, projects created, pipeline runs, alerts triggered.
- **Recent activity**: Latest pipeline runs with status indicators.
- **Sidebar navigation**: Projects, Hosts, Pipelines, Monitoring, Alerts, Logs, Settings, Documentation.

8.4 Projects Page

The projects listing page displays all projects for the current organisation:

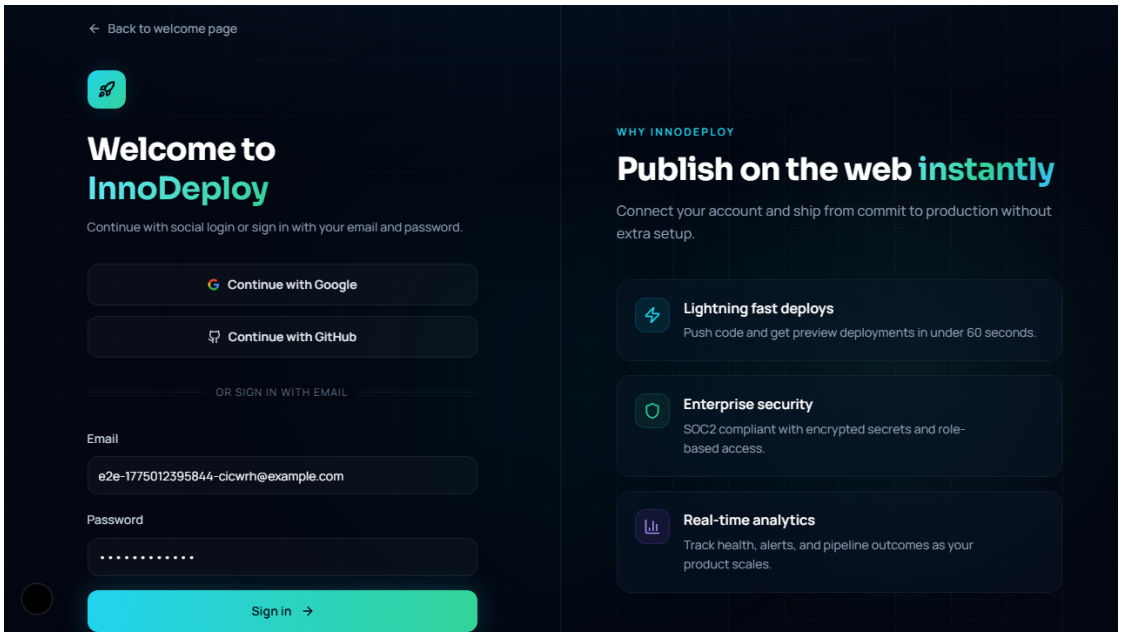

Figure 8.3: Projects Page – Project List with Status

Features:

- Project cards showing name, repository URL, status, and last deploy time.
- Quick-action buttons for deployment, pipeline trigger, and settings.
- Environment badges showing configured deployment targets.

8.5 Key UI Component Summary

Table 8.1: Dashboard UI Components

Page/Component	**Description**
Landing Page	Marketing page with hero section, feature cards, pricing, and testimonials
Dashboard	Admin overview with stats, trends, and recent activity
Projects List	Card grid of all projects with status and quick actions
Project Detail	Environments, secrets, deployment history, and settings
New Project	Form wizard for creating a project from a Git repository
Pipeline Detail	Step-by-step execution view with live log output and status
Hosts Page	Host list with SSH status, resource usage, and container info
Monitoring	Real-time charts (CPU, memory, disk, latency) using Recharts
Logs	Terminal-style log viewer (xterm.js) with filters and search
Alerts	Alert list with severity badges, acknowledge/resolve actions
Settings	Organisation profile, members, notifications, Docker registry, API keys
Admin Dashboard	System-wide stats with user management (owner/admin only)
Docs	Built-in documentation viewer
Support	Help and support page

Chapter 9

Deployment

9.1 Deployment Diagram

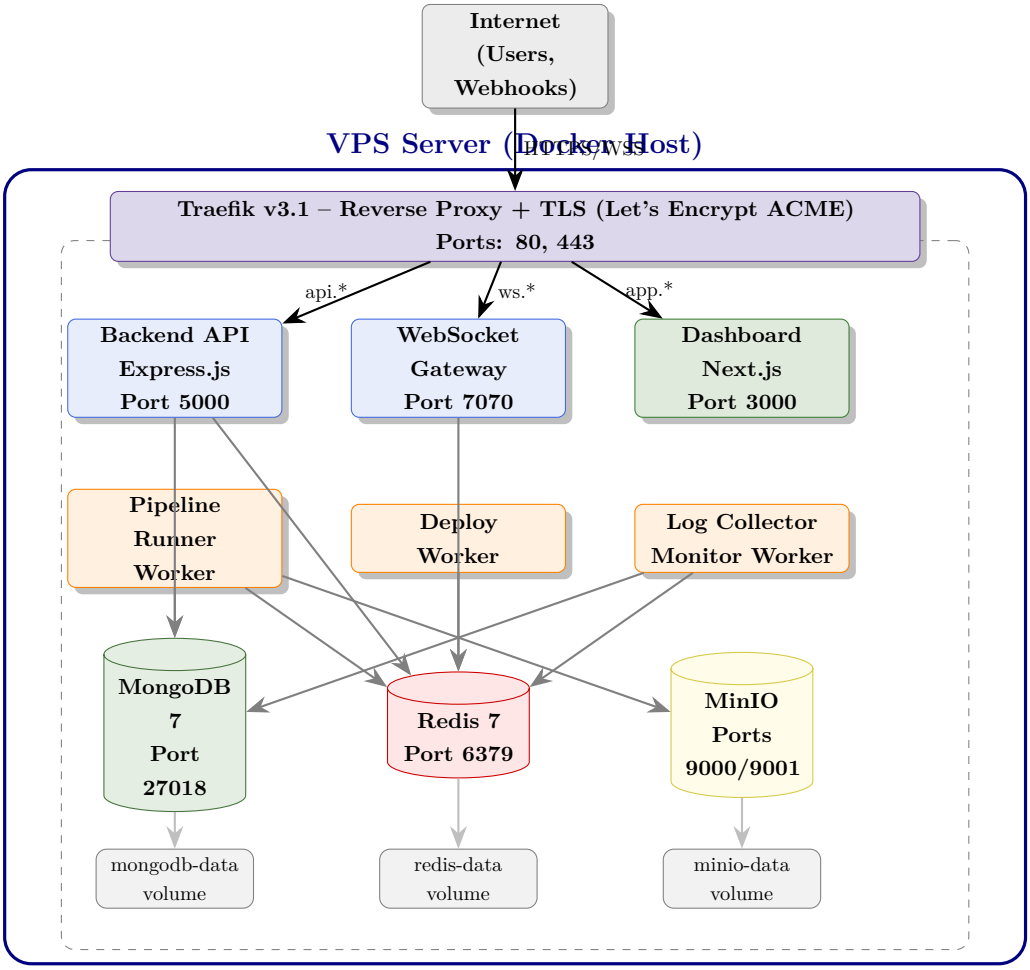

Figure 9.1: UML Deployment Diagram

9.2 Docker Compose Configuration

The entire platform is orchestrated using Docker Compose with 8 services:

Table 9.1: Docker Compose Services

Service	**Image/Base**	**Role**
traefik	traefik:v3.1	Reverse proxy with auto-TLS via Let's Encrypt
backend	Custom (Dockerfile)	Express.js API server
websocket	Custom (Dockerfile)	WebSocket gateway for real-time updates
dashboard	Custom (Dockerfile)	Next.js frontend application
mongodb	mongo:7	Primary database
redis	redis:7	Cache, queues, pub/sub, token storage
minio	minio	S3-compatible artifact storage
pipeline-runner	Custom (Dockerfile)	Background pipeline execution worker
deploy-worker	Custom (Dockerfile)	Background deployment execution worker

9.3 Traefik Configuration

Traefik provides:

- **Automatic HTTPS**: Let's Encrypt ACME with HTTP-01 challenge.
- **Service discovery**: Docker labels for automatic route registration.
- **Rate limiting**: Middleware for API endpoint protection.
- **Domain routing**: `api.*` → backend, `ws.*` → websocket, `app.*` → dashboard.
- **WebSocket support**: Transparent upgrade for the WebSocket gateway.

9.4 Environment Variables

Table 9.2: Key Environment Variables

Variable	Default	Purpose
PORT	5000	Backend API port
MONGO_URI	localhost:27017	MongoDB connection string
REDIS_URL	localhost:6379	Redis connection string
JWT_SECRET	–	JWT signing secret
JWT_REFRESH_SECRET	–	Refresh token signing secret
CLIENT_URL	–	CORS allowed origin
GOOGLE_CLIENT_ID	–	Google OAuth credentials
GITHUB_CLIENT_ID	–	GitHub OAuth credentials
WS_PORT	7070	WebSocket gateway port
PIPELINE_RUNNER_CONCURRENCY	1	Parallel pipeline slots
PIPELINE_STEP_TIMEOUT_MS	600000	Step execution timeout
LOG_RETENTION_DAYS	30	Log auto-expiration period

9.5 Deployment Strategies Supported

InnoDeploy supports four deployment strategies for application delivery:

Table 9.3: Deployment Strategies

Strategy	**Description**	**Downtime**
Rolling	Incrementally replace old containers with new ones, performing health checks between each swap.	Zero
Blue-Green	Spin up a complete "green" environment, then switch traffic from "blue" to "green" atomically.	Near-zero
Canary	Gradually route increasing traffic percentages to the new version while monitoring for errors.	Zero
Recreate	Stop all old containers, then start new ones. Simple but causes brief downtime. Used for stateful services and rollbacks.	Brief

Chapter 10

Testing

10.1 Testing Strategy

InnoDeploy follows a multi-layer testing strategy:

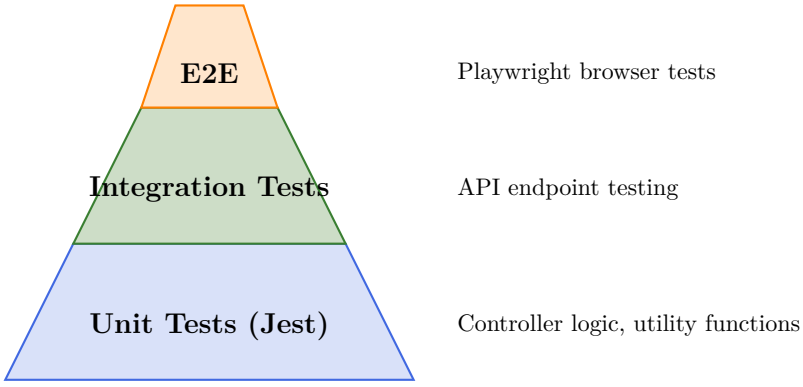

Figure 10.1: Testing Pyramid

10.2 Unit Testing

Unit tests are implemented using **Jest** for the backend:

- Controller functions tested in isolation with mocked models and services.
- Utility functions (encryption, token generation, validation) tested with edge cases.
- Test command: `npm test` with open handle detection.

10.3 Integration Testing

Integration tests verify API endpoints with a test database:

- HTTP requests against the Express app with supertest.
- Database seeding and cleanup between test suites.
- Authentication flow testing (register, login, refresh, logout).
- Project CRUD with environment and secret management.

10.4 End-to-End Testing

E2E tests use **Playwright** for browser-based testing of the dashboard:

- **Auth flows**: Registration with organisation creation, login with existing credentials.
- **Project management**: Creating projects, viewing project lists.
- **Pipeline execution**: Triggering pipelines from the project detail page.
- Chromium-based test execution with screenshot capture on failure.
- Configuration: `playwright.config.ts` with base URL and timeout settings.

10.5 Pipeline End-to-End Testing

A dedicated pipeline E2E test (`pipeline-e2e.js`) validates the complete pipeline lifecycle:

1. Register a test user and create an organisation.
2. Create a project linked to a test repository.
3. Trigger a pipeline run with custom steps.
4. Poll the pipeline status until completion.
5. Verify step outputs and final status.
6. Clean up test data.

Chapter 11

Conclusion

11.1 Summary of Contributions

This project has successfully designed and implemented **InnoDeploy**, a comprehensive multi-tenant DevOps SaaS platform that addresses the fragmentation and complexity challenges in modern CI/CD tooling. The key contributions include:

1. **Unified Platform**: A single platform consolidating CI/CD pipelines, deployment orchestration, monitoring, alerting, log aggregation, and secret management – eliminating the need for multiple disparate tools.

2. **Multi-Tenant Architecture**: Organisation-scoped resource isolation with four-tier RBAC (Owner, Admin, Developer, Viewer), plan-based limits, and complete audit trail with 90-day retention.

3. **Pipeline Execution Engine**: Docker-container-isolated step execution with configurable timeouts, retries, and real-time progress streaming via SSE and WebSocket.

4. **Multi-Strategy Deployment**: Support for rolling, blue-green, canary, and recreate deployment strategies with SSH-based execution on user-managed hosts.

5. **Real-Time Monitoring**: WebSocket-based metric streaming with configurable alert thresholds and multi-channel notification dispatch (email, Slack, Discord, Expo push, webhooks).

6. **Modern Dashboard**: A responsive Next.js dashboard with dark theme, internationalisation support, real-time data visualization, terminal-style log viewer, and Monaco code editor integration.

7. **Developer-Friendly CLI**: A command-line tool providing full platform access with interactive prompts and automatic token refresh.

8. **Production-Ready Infrastructure**: Docker Compose orchestration with Traefik reverse proxy, automatic TLS certificates, and persistent data volumes.

11.2 Technical Achievements

- **9 data models** with comprehensive indexing and TTL-based data lifecycle management.
- **60+ REST API endpoints** with Zod schema validation and role-based access control.
- **4 background workers** (pipeline runner, deploy worker, monitor worker, log collector) with Redis-backed job queues.
- **15 service modules** handling notifications, queues, event streaming, and infrastructure integration.
- **3 webhook receivers** (GitHub, GitLab, Bitbucket) with cryptographic signature verification.
- **Multi-layer testing** with unit tests (Jest), integration tests, and E2E tests (Playwright).

11.3 Limitations

- **Single-node workers**: Pipeline and deploy workers run as single instances; horizontal scaling requires manual sharding.
- **Docker-only containers**: Pipeline steps currently only support Docker-based execution environments.
- **No built-in Git server**: The platform relies on external Git providers (GitHub, GitLab, Bitbucket).

11.4 Future Work

1. **Kubernetes support**: Extend deployment targets to include Kubernetes clusters alongside Docker hosts.

2. **Auto-scaling workers**: Implement dynamic worker scaling based on queue depth using Kubernetes HPA or Docker Swarm.

3. **Plugin system**: Allow users to extend pipeline steps with custom plugins for specific build tools or deployment targets.

4. **AI-powered insights**: Integrate machine learning for anomaly detection in metrics and predictive alerting.

5. **Mobile application**: Develop native mobile apps for monitoring and alert management on-the-go.

6. **GitOps integration**: Support declarative deployment configuration synced from Git repositories.
7. **Multi-region deployment**: Enable geographic distribution of deployments across multiple VPS regions.

11.5 Final Remarks

InnoDeploy demonstrates that a well-architected full-stack application can provide enterprise-grade DevOps capabilities in a unified, self-hosted package. The combination of real-time communication, containerised execution, multi-strategy deployment, and comprehensive observability creates a platform that is both powerful for production use and educational as an engineering project.

The project showcases modern best practices across the entire stack: event-driven architecture, JWT-based authentication with server-side revocation, Zod schema validation, Redis pub/sub for real-time features, and Docker Compose for reproducible deployment – making it a comprehensive reference implementation for multi-tenant SaaS platform engineering.

Appendix A

Project Directory Structure

```
InnoDeploy/
|-- backend/
|   |-- Dockerfile
|   |-- package.json
|   `-- src/
|       |-- app.js              # Express setup
|       |-- server.js           # Bootstrap & shutdown
|       |-- config/
|       |   |-- db.js           # MongoDB connection
|       |   `-- redis.js        # Redis connection
|       |-- controllers/        # 11 controllers
|       |-- middleware/          # Auth, audit, validate, error
|       |-- models/             # 9 Mongoose models
|       |-- routes/             # 12 route files
|       |-- services/           # 15 service modules
|       |-- utils/              # Encryption, helpers
|       `-- workers/            # Worker entry points
|-- dashboard/
|   |-- Dockerfile
|   |-- package.json
|   |-- app/                    # Next.js pages (10 routes)
|   |-- components/             # 16 component directories
|   |-- hooks/                  # 3 custom hooks
|   |-- lib/                    # API client, socket, utils
|   |-- store/                  # Zustand auth store
|   |-- types/                  # TypeScript interfaces
|   `-- tests/                  # Playwright E2E tests
|-- cli/
|   |-- package.json
|   |-- bin/innodeploy.js       # CLI entry point
|   `-- src/                    # Config & prompts
|-- docker/
|   `-- docker-compose.yml      # 8-service orchestration
`-- docs/
    |-- architecture.md
    |-- PROJECT_STRUCTURE.md
    `-- README.md
```

Appendix B

Glossary

API Application Programming Interface – a set of protocols for building software.

RBAC Role-Based Access Control – restricting system access based on user roles.

CI/CD Continuous Integration / Continuous Deployment – automated build and release pipeline.

JWT JSON Web Token – compact, URL-safe token for claims transmission.

SSE Server-Sent Events – HTTP-based one-way real-time data push.

WebSocket Full-duplex communication protocol over a single TCP connection.

SaaS Software as a Service – cloud-delivered application model.

TLS Transport Layer Security – cryptographic protocol for secure communication.

ACME Automatic Certificate Management Environment – protocol for TLS certificate automation.

ODM Object-Document Mapper – maps objects to document database records.

TTL Time to Live – automatic expiration period for data records.

Pub/Sub Publish/Subscribe – messaging pattern for decoupled communication.

SSH Secure Shell – protocol for secure remote server access.

OAuth Open Authorization – delegated authentication protocol.

HMAC Hash-based Message Authentication Code – cryptographic signature for message integrity.

CORS Cross-Origin Resource Sharing – browser security mechanism for cross-domain requests.